

LEDGERLY

Technical report · Mini-project 3 · Flutter Week 08 Part 2

1. Product direction

Ledgerly is an offline-first expense tracker for VKU students and club managers. The product intentionally keeps a human, tactile feel: paper-like warm surfaces, cobalt navigation, compact cards, visible review steps and no generated content pretending to be a financial decision. OCR accelerates data entry; the user remains the final verifier.

2. Architecture and data flow



Flutter presentation uses Material 3 with responsive NavigationBar / NavigationRail, a GoRouter ShellRoute for URL-addressable destinations, Riverpod for app state, SQLite for local persistence, and CustomPainter for the two charts. Native battery information is a small MethodChannel proof of Android Kotlin and iOS Swift interop.

Implemented layers

Layer	Implementation	Responsibility
UI	LedgerlyApp, AppShell, screens, widgets	Responsive layout, theme, dark mode, dialogs and review sheet
State	ReceiptController + Riverpod	Load, add, edit, delete, search and filter receipts
Domain	Receipt, ReceiptParser, parseVndInput	Normalized data and deterministic OCR extraction
Services	OcrService, ReceiptRepository, DeviceInfoService	ML Kit, SQLite/memory fallback, MethodChannel
Navigation	core/router.dart	ShellRoute, guarded category query and receipt detail path
Screens/widgets	screens/ and widgets/	Pages, review sheet, reusable cards and CustomPainter charts

3. OCR and parsing contract

Text recognition runs on-device through `google_mlkit_text_recognition`. The review form is deliberately mandatory: confidence is not silently treated as truth, and Vietnamese number grouping is normalized before persistence.

Field	Pattern / strategy	Fallback and review behavior
Total	<code>(?i)(total amount tổng thành tiền)\s*[:\s]?[0-9][0-9.,]*)</code>	Normalize dots/commas/spaces; reject zero or invalid input
Date	<code>\b(20\d{2})[-./](\d{1,2})[-./](\d{1,2})\b</code>	Also accepts dd/mm/yyyy; future dates are blocked
Merchant	First non-empty OCR line not claimed as total/date	User can edit merchant in the review sheet
Category	Keyword buckets: food, transport, shopping, bills, other	Other is safe default; user can change category

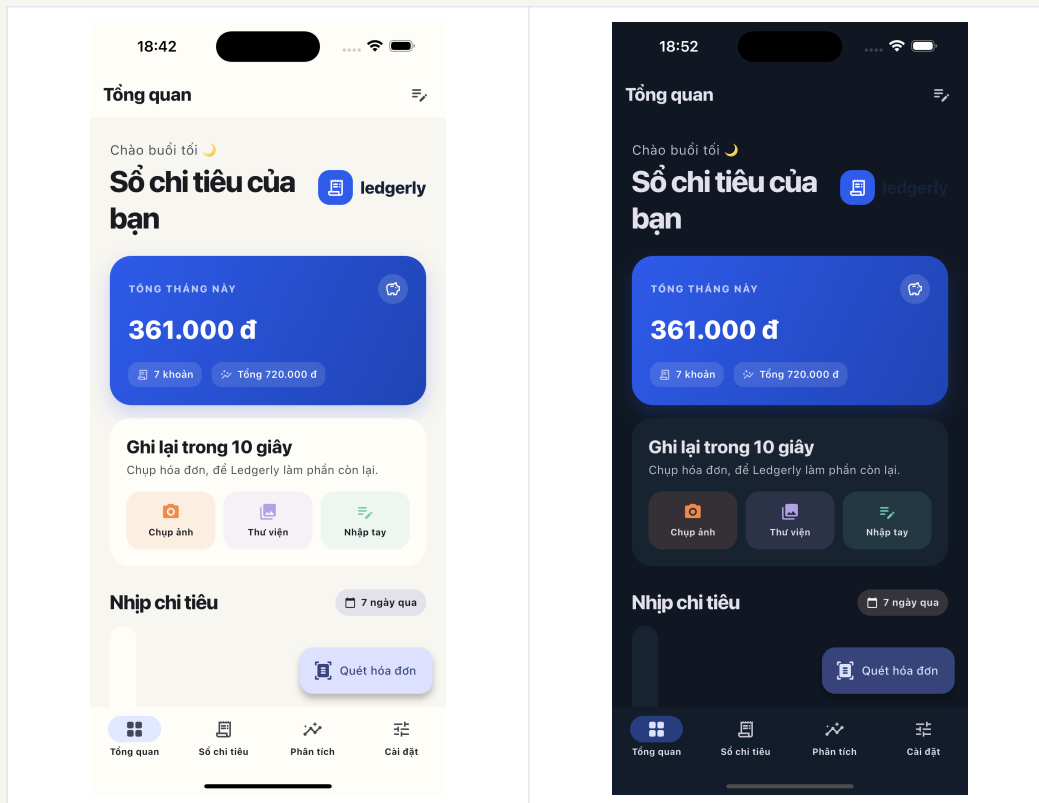
4. Navigation, forms and native bridge

Sprint capability	Delivered behavior
GoRouter	ShellRoute with <code>/</code> , <code>/receipts</code> , <code>/insights</code> , <code>/settings</code> ; guarded category query and <code>/receipts/:receiptId</code> detail path. Fade/slide transitions preserve a calm deep-link flow.
Form safety	Autovalidation after interaction, positive VND validation, focus traversal, date picker with future-date guard, save confirmation <code>SnackBar</code> .
MethodChannel	<code>vn.edu.vku/device_info</code> returns battery percentage from Kotlin <code>BatteryManager</code> on Android and <code>UIDevice</code> on iOS; unsupported platforms fall back gracefully.
Adaptive UI	Bottom navigation on compact widths; side rail and wider two-column content when space allows.

5. Visual language

Light mode	Dark mode	Interaction principles
Warm paper #F8F5EE Cobalt #2857A6 Orange #E97845	Ink surfaces with preserved cobalt/orange accents High-contrast text and chart colors	Review before commit Short actions Explicit empty/error states Touch targets kept generous

6. Runtime screenshots



The screenshots were captured from the iPhone 15 Pro Max Simulator after a successful runtime launch. The light and dark surfaces keep the same information hierarchy, chart accent colors and navigation affordances while adapting contrast and elevation.

7. Rubric and verification matrix

Rubric	Evidence in project	Status
On-device OCR & heuristics · 3.5	Camera/gallery, ML Kit, parser, review form	Implemented
Custom canvas visualization · 2.5	Animated donut and weekly bar painters	Implemented
State management & DB · 2.0	Riverpod StateNotifier, SQLite CRUD, memory fallback	Implemented
UI/UX polish · 1.0	Material 3, dark mode, responsive shell, correction dialog	Implemented
Deliverables & report · 1.0	This PDF, README, demo script, APK, simulator build, license and CI workflow	Ready; student video/GitHub supplied

Verification run

flutter analyze --no-fatal-infos: no issues found. flutter test: 4 tests passed, covering currency normalization, parser extraction, dashboard rendering and nested route registry. flutter build apk --release --no-pub: passed. flutter build appbundle --release --no-pub: passed. flutter build ios --simulator --no-codesign --no-pub: passed. iPhone Simulator smoke launch passed; physical-phone testing was confirmed by the student.

8. Known limitations and handoff

The current local APK uses the debug signing fallback because no private keystore was supplied. Before store submission, create the free private keystore described in docs/DEPLOYMENT.md, wire it through key.properties/Gradle, and rebuild. OCR quality depends on focus, lighting and the Latin model; receipts with unusual layouts still need manual correction. Video demo and GitHub handoff are supplied separately by the student.

Build artifacts

Artifact	Path
Android release-mode APK	build/app/outputs/flutter-apk/app-release.apk
Google Play AAB	build/app/outputs/bundle/release/app-release.aab
iOS Simulator app	build/ios/iphonesimulator/Runner.app
Demo script	docs/DEMO_SCRIPT.md